MODUL PRAKTIKUM BASIS DATA

Praktikum 2

Database dan Table Creation

Program Studi	S1 Teknik Informatika
Mata Kuliah	Basis Data
Praktikum / Minggu	Praktikum 2 · Minggu 2 (mendampingi kuliah P02 Relational Model)
DBMS	MySQL 8.x / MariaDB 10.x (lokal)
Database	db_akademik
Nama / NIM	
Kelas / Tanggal	
Dosen / Asisten	Teuku Mufizar, M.Kom

1. Judul Praktikum

Praktikum 2 — Database dan Table Creation

2. Minggu

Minggu ke	2
Kuliah pendamping	P02 Relational Model
Durasi	150–170 menit di laboratorium + pengerjaan lanjutan mandiri
Bobot nilai akhir	1%

3. CPMK

- CPMK-1: Mampu menjelaskan konsep sistem basis data, komponen dan fungsi DBMS, arsitektur konseptual, model data, serta model relasional (relation, tuple, attribute, domain, key) beserta perannya dalam sistem Informatika, dengan mengaitkannya pada minimal satu contoh aplikasi nyata.

4. Sub-CPMK

- S2: Menjelaskan arsitektur basis data secara konseptual, perbedaan schema dan instance, serta komponen model relasional (relation, tuple, attribute, domain, key).
- S9: Membuat database dan tabel menggunakan DDL (CREATE, ALTER, DROP) beserta integrity constraint, dan menguji constraint.

5. Tujuan

Setelah praktikum ini mahasiswa mampu:

- Menerjemahkan kebutuhan data menjadi rancangan kolom: nama, tipe data, ukuran, dan aturan (PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT).
- Membuat tabel `program_studi`, `dosen`, dan `mahasiswa` pada `db_akademik` dengan `CREATE TABLE` yang benar.
- Membuktikan bahwa constraint bekerja dengan menguji data valid, PK duplikat, UNIQUE duplikat, dan NOT NULL yang dilanggar.
- Membedakan skema (relation schema) dari instance (isi tabel) dan memetakan istilah relation, tuple, attribute, domain ke objek MySQL/MariaDB.

6. Kompetensi

Kompetensi yang dilatihkan (indikator keberhasilan):

- Memilih tipe data yang sesuai domain atribut dan mampu menjelaskan alasannya (indikator: setiap kolom punya justifikasi tertulis).
- Menulis `CREATE TABLE` lengkap dengan constraint tanpa error sintaks (indikator: `DESCRIBE` menampilkan kunci dan nullability sesuai rancangan).
- Membaca pesan error 1062, 1048, 1364, 1265 dan menghubungkannya dengan constraint yang dilanggar.

7. Problem Scenario

Tim pengembang Sistem Akademik Informatika sudah menyiapkan server dan database kosong `db_akademik` (Praktikum 1). Pekan ini Anda ditugaskan membuat tiga tabel dasar: `program_studi`, `dosen`, dan `mahasiswa`. Bagian akademik menyerahkan kebutuhan data dalam bentuk kalimat (lihat bagian 10), bukan dalam bentuk tabel. Anda yang harus memutuskan nama kolom, tipe data, ukuran, dan aturan yang menjaga kualitas data.

Dua masalah nyata harus dicegah sejak tabel dibuat: dua mahasiswa dengan NIM yang sama, dan dua dosen dengan alamat email yang sama. Selain itu, nama mahasiswa tidak boleh kosong, sedangkan email boleh kosong karena tidak semua mahasiswa baru memilikinya. Setelah tabel jadi, Anda wajib membuktikan bahwa aturan-aturan itu benar-benar ditegakkan oleh DBMS, bukan sekadar niat.

8. Dasar Teori

Relational model. Data disimpan sebagai *relation* yang tampil sebagai tabel. Baris disebut *tuple* (di MySQL: row/record), kolom disebut *attribute* (column/field), dan himpunan nilai yang sah untuk suatu atribut disebut *domain*. Domain diwujudkan dengan tipe data ditambah constraint.

Skema vs instance. *Skema* adalah rancangan yang jarang berubah, misalnya `mahasiswa(nim, nama, angkatan, ...)`. *Instance* adalah isi tabel pada suatu saat dan terus berubah. `CREATE TABLE` mendefinisikan skema; `INSERT` mengubah instance.

Key. *Superkey* adalah himpunan atribut yang mengidentifikasi tuple secara unik. *Candidate key* adalah superkey minimal. *Primary key* adalah satu candidate key yang dipilih sebagai identitas utama (tidak boleh NULL, tidak boleh kembar). Candidate key lain dijaga dengan `UNIQUE`. Pada `mahasiswa`, `nim` dan `email` sama-sama candidate key; `nim` dipilih sebagai PK karena wajib ada dan stabil.

Tipe data yang dipakai pada praktikum ini.

- `CHAR(n)`: panjang tetap, cocok untuk kode seragam (kode prodi 3 huruf, NIDN 10 digit).
- `VARCHAR(n)`: panjang berubah sampai n, cocok untuk nama dan email.
- `SMALLINT` / `TINYINT` / `INT`: bilangan bulat, pilih yang cukup untuk rentang nilainya.
- `DATE`: tanggal `YYYY-MM-DD`; `ENUM(...)`: satu nilai dari daftar tertutup.
- Identitas seperti NIM atau NIDN disimpan sebagai teks (`CHAR`), bukan angka, karena tidak dihitung dan angka nol di depan tidak boleh hilang.

Constraint kolom. `NOT NULL` melarang nilai kosong. `DEFAULT x` mengisi otomatis bila kolom tidak disebut saat `INSERT`. `UNIQUE` melarang nilai kembar (nilai NULL tidak dianggap kembar). `PRIMARY KEY` = `NOT NULL` + `UNIQUE` + identitas utama tabel. Setiap pelanggaran ditolak DBMS dengan error.

```
CREATE TABLE nama_tabel (
  kolom1 TIPE NOT NULL,
  kolom2 TIPE NULL DEFAULT nilai,
  PRIMARY KEY (kolom1),
  UNIQUE (kolom2)
) ENGINE=InnoDB;
```

Kerangka CREATE TABLE. Tanda koma memisahkan definisi; tidak ada koma setelah definisi terakhir.

```
INSERT INTO nama_tabel (kolom1, kolom2)
VALUES ('a', 'b'), ('c', NULL);
```

INSERT dipakai di sini hanya untuk menguji constraint; pembahasan lengkap DML ada pada Praktikum 12.

Foreign key sengaja **belum** dipasang pada tabel-tabel ini agar urutan pembuatan tidak menjadi hambatan. Foreign key dibahas mulai Praktikum 4 dan diterapkan pada Praktikum 7; pada bagian Challenge Anda mencobanya lebih awal dengan `ALTER TABLE`.

9. Tools

- MySQL Community Server 8.x **atau** MariaDB Server 10.x, terpasang lokal
- Klien command-line `mysql`/`mariadb` (utama untuk menjalankan skrip `.sql`)
- GUI opsional: MySQL Workbench, DBeaver, atau HeidiSQL
- Editor teks (VS Code/Notepad++) untuk menyimpan `praktikum_02.sql`
- Folder kerja berisi 01_create_database.sql, 02_create_tables.sql, 03_insert_data.sql

10. Database Schema

Kebutuhan data dari bagian akademik (belum berupa tabel; Anda yang merancang kolom dan tipenya). Tuliskan rancangan Anda pada tahap Design **sebelum** menulis SQL.

program_studi. Setiap prodi punya kode 3 karakter (contoh IF1, SI1, DS1) sebagai identitas. Nama prodi wajib diisi dan tidak boleh ada dua prodi bernama sama. Nama fakultas wajib diisi.

dosen. Identitas dosen adalah NIDN (10 digit). Nama wajib. Email boleh kosong, tetapi bila diisi tidak boleh dipakai dua dosen. Jabatan salah satu dari Asisten Ahli, Lektor, Lektor Kepala, Guru Besar; bila tidak dinyatakan, dianggap Asisten Ahli. Setiap dosen wajib tercatat pada satu prodi (kode prodi).

mahasiswa. Identitas mahasiswa adalah NIM (teks, paling banyak 10 karakter). Nama wajib (sampai 80 karakter). Angkatan (tahun) wajib. Email boleh kosong, bila diisi harus unik. Tanggal lahir boleh kosong. Status mahasiswa salah satu dari Aktif, Cuti, Lulus dan otomatis Aktif bila tidak dinyatakan. Kode prodi wajib. Dosen wali (NIDN) boleh kosong.

Catatan: rancangan akhir pada kunci sama dengan bagian tabel-tabel tersebut pada `02_create_tables.sql`, tanpa CHECK dan foreign key (keduanya baru dibahas pada praktikum berikutnya).

11. Sample Data

Praktikum ini dimulai dari database kosong. Data uji berikut Anda masukkan sendiri pada Latihan 5 (tulis `INSERT`-nya sendiri).

program_studi (3 baris): IF1 = S1 Informatika; SI1 = S1 Sistem Informasi; DS1 = S1 Sains Data; ketiganya berfakultas seperti yang tertera pada Latihan 5.

dosen (2 baris): 0410018801 Dr. Rina Wulandari, M.Kom. (Lektor Kepala, IF1, email rina.w@kampus.test); 0412129308 Ahmad Fauzi, M.T. (jabatan tidak dinyatakan, IF1, tanpa email).

mahasiswa (3 baris): 24010002 Sari Lestari; 25030003 Budi Santoso (tanpa email); 25010011 Reza Firmansyah. Angka lengkap tertera pada Latihan 5.

12. Langkah Kerja

Ikuti alur **Problem → Think → Design → Write SQL → Execute → Observe → Interpret → Modify → Test**. Tahap *Think/Design/Write SQL* harus Anda kerjakan sendiri di kertas atau berkas `.sql` **sebelum** melihat contoh pada bagian 13.

No	Tahap	Kegiatan mahasiswa	Bukti
1	Problem	Baca skenario dan kebutuhan data (bagian 10). Tulis dua kesalahan data yang harus dicegah oleh tabel `mahasiswa` dan dua yang harus dicegah oleh tabel `dosen`.	Daftar 4 risiko
2	Think	Untuk setiap atribut, tebak: teks atau angka? panjang tetap atau berubah? boleh kosong? harus unik? Tulis tebakan pada tabel kolom (nama, tipe, NULL/NOT NULL, unik).	Tabel kolom draf
3	Design	Tetapkan candidate key setiap tabel lalu pilih primary key. Jelaskan mengapa NIM lebih layak menjadi PK daripada email, dan mengapa NIM disimpan sebagai `CHAR`, bukan `INT`.	Justifikasi key + tipe
4	Write SQL	Tulis sendiri `CREATE TABLE` ketiga tabel di `praktikum_02.sql` dengan urutan pembuatan yang Anda pilih. Jangan membuka kunci atau menyalin dari `02_create_tables.sql`.	Berkas .sql
5	Execute	Jalankan berkas melalui klien (`source praktikum_02.sql;`). Perbaiki setiap error sintaks satu per satu dan catat nomor error yang Anda temui.	Tangkapan layar + log error
6	Observe	Jalankan `SHOW TABLES;` dan `DESCRIBE` untuk tiap tabel. Bandingkan kolom Key dan Null dengan tabel kolom draf Anda.	Tabel draf vs hasil

No	Tahap	Kegiatan mahasiswa	Bukti
7	Interpret	Jelaskan arti nilai `PRI`, `UNI`, `YES`/`NO` pada hasil `DESCRIBE`, dan mengapa `email` boleh NULL namun tetap `UNI`.	Jawaban bagian 17
8	Modify	Ubah rancangan: bagian akademik meminta email mahasiswa wajib diisi. Coba ubah tabel Anda, catat hasilnya, dan tentukan langkah aman yang seharusnya ditempuh.	Catatan perubahan + alasan
9	Test	Masukkan data valid, lalu sengaja langgar tiap constraint (PK kembar, email kembar, nama kosong, status di luar daftar). Catat kode error dan constraint mana yang bereaksi.	Tabel uji: skenario, hasil, error

13. SQL Example

Contoh berikut sudah dijalankan pada MariaDB 10.11 (kompatibel MySQL 8.x kecuali disebutkan). Bandingkan dengan jawaban Anda **setelah** mencoba sendiri.

1. Membuat tabel contoh ruangan (kasus lain, bukan tabel latihan)

```
CREATE TABLE ruangan (
  kode_ruang  VARCHAR(10) NOT NULL,
  nama_gedung VARCHAR(40) NOT NULL,
  kapasitas   SMALLINT    NOT NULL,
  proyektor   ENUM('Ada','Tidak') NOT NULL DEFAULT 'Tidak',
  catatan     VARCHAR(100) NULL,
  PRIMARY KEY (kode_ruang)
) ENGINE=InnoDB;
```

Keluaran:

```
(tidak ada keluaran / Query OK)
```

Pola: kolom identitas NOT NULL + PRIMARY KEY, kolom wajib NOT NULL, kolom opsional NULL, kolom yang punya nilai bawaan memakai DEFAULT.

2. Membaca struktur tabel

```
DESCRIBE ruangan;
```

Keluaran:

```
+-------------+--------------------+------+-----+---------+-------+
| Field       | Type               | Null | Key | Default | Extra |
+-------------+--------------------+------+-----+---------+-------+
| kode_ruang  | varchar(10)        | NO   | PRI | NULL    |       |
| nama_gedung | varchar(40)        | NO   |     | NULL    |       |
| kapasitas   | smallint(6)        | NO   |     | NULL    |       |
| proyektor   | enum('Ada','Tidak') | NO   |     | Tidak   |       |
| catatan     | varchar(100)       | YES  |     | NULL    |       |
+-------------+--------------------+------+-----+---------+-------+
```

Kolom Key berisi PRI untuk PK. Kolom Null menunjukkan boleh kosong atau tidak; Default menunjukkan nilai bawaan.

3. Mengisi data valid (kolom proyektor sengaja tidak disebut)

```
INSERT INTO ruangan (kode_ruang, nama_gedung, kapasitas)
VALUES ('R101', 'Gedung A', 40), ('LAB1', 'Gedung B', 30);
SELECT * FROM ruangan;
```

Keluaran:

```
+------------+-------------+-----------+-----------+---------+
| kode_ruang | nama_gedung | kapasitas | proyektor | catatan |
+------------+-------------+-----------+-----------+---------+
| LAB1       | Gedung B    |        30 | Tidak     | NULL    |
| R101       | Gedung A    |        40 | Tidak     | NULL    |
+------------+-------------+-----------+-----------+---------+
```

Kolom proyektor dan catatan terisi otomatis: DEFAULT dan NULL.

4. Menguji PRIMARY KEY: kode_ruang kembar

```
INSERT INTO ruangan (kode_ruang, nama_gedung, kapasitas)
VALUES ('R101', 'Gedung C', 25);
```

Keluaran:

```
ERROR 1062 (23000): Duplicate entry 'R101' for key 'PRIMARY'
```

Error 1062: DBMS menolak baris kedua karena identitas kembar. Tabel tetap berisi 2 baris.

5. Melihat DDL yang tersimpan di server

```
SHOW CREATE TABLE ruangan\G
```

Keluaran:

```
*************************** 1. row ***************************
       Table: ruangan
Create Table: CREATE TABLE `ruangan` (
  `kode_ruang` varchar(10) NOT NULL,
  `nama_gedung` varchar(40) NOT NULL,
  `kapasitas` smallint(6) NOT NULL,
  `proyektor` enum('Ada','Tidak') NOT NULL DEFAULT 'Tidak',
  `catatan` varchar(100) DEFAULT NULL,
  PRIMARY KEY (`kode_ruang`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
```

Server menuliskan ulang definisi tabel lengkap dengan character set dan engine. Berguna untuk memeriksa hasil rancangan.

6. Membersihkan tabel contoh

```
DROP TABLE ruangan;
SHOW TABLES;
```

Keluaran:

```
(tidak ada keluaran / Query OK)
```

Tabel contoh dibuang agar `db_akademik` hanya berisi tabel latihan. Ingat: DROP TABLE menghapus struktur dan seluruh datanya.

14. Student Exercises

Kerjakan **11 latihan** berikut secara mandiri; tulis jawaban di `praktikum_02.sql` dengan komentar nomor soal. Jangan hanya menulis query: **jelaskan** apa yang Anda harapkan sebelum menjalankan, lalu bandingkan dengan hasilnya.

No	Soal / masalah	Petunjuk berpikir
1	Rancang dan buat tabel `program_studi` berdasarkan kebutuhan data. Pilih sendiri nama kolom, tipe, dan ukuran; kode prodi adalah identitas, nama prodi tidak boleh kembar. Jelaskan mengapa tipe kode prodi Anda pilih.	Kode berformat seragam 3 karakter. Kolom mana yang wajib, mana yang unik?
2	Buat tabel `dosen`. NIDN adalah identitas, email boleh kosong tetapi tidak boleh kembar, jabatan hanya boleh empat nilai dan otomatis "Asisten Ahli", dan setiap dosen wajib punya kode prodi. Foreign key belum dipasang.	Jabatan adalah domain berdaftar tertutup: pikirkan ENUM dan DEFAULT. Email: NULL + UNIQUE.
3	Buat tabel `mahasiswa` sesuai kebutuhan data (NIM identitas; email opsional tetapi unik; status default Aktif; tanggal lahir opsional; kode prodi	Tahun 4 digit muat di SMALLINT. Kolom mana yang boleh NULL?

No	Soal / masalah	Petunjuk berpikir
	wajib; dosen wali opsional). Tentukan tipe angkatan yang paling hemat namun cukup untuk tahun.	
4	Buktikan dengan satu query ke `information_schema` bahwa kolom kunci ketiga tabel sesuai rancangan: tampilkan nama tabel, nama kolom, jenis kunci (PRI/UNI), dan status nullable, hanya untuk kolom yang memiliki kunci pada database aktif.	Tabel `information_schema.columns`; kolom `column_key`, `is_nullable`; gunakan `DATABASE()`.
5	Masukkan data uji yang valid: 3 program studi (IF1 S1 Informatika; SI1 S1 Sistem Informasi; DS1 S1 Sains Data, fakultas masing-masing Fakultas Teknik dan Informatika, Fakultas Teknik dan Informatika, Fakultas Sains dan Teknologi), 2 dosen (0410018801 Dr. Rina Wulandari, M.Kom., email rina.w@kampus.test, Lektor Kepala, IF1; 0412129308 Ahmad Fauzi, M.T., tanpa email, jabatan tidak disebut, IF1), dan 3 mahasiswa (24010002 Sari Lestari, 2024, IF1; 25030003 Budi Santoso, 2025, DS1, tanpa email; 25010011 Reza Firmansyah, 2025, IF1). Tampilkan mahasiswa dan jabatan dosen yang terisi otomatis sebagai bukti DEFAULT.	Kolom yang tidak disebut pada INSERT akan diisi DEFAULT atau NULL. Sebutkan daftar kolom secara eksplisit.
6	Uji primary key: coba tambahkan mahasiswa baru bernama "Tiara Ayu" (angkatan 2025, prodi IF1) dengan NIM 24010002 yang sudah dipakai Sari Lestari. Prediksikan hasilnya sebelum menjalankan, lalu tunjukkan bahwa data lama tetap utuh.	Apa yang ditolak: nama, NIM, atau seluruh baris?
7	Uji UNIQUE: tambahkan mahasiswa dengan NIM baru 25010021 (nama "Rangga Wijaya", 2025, IF1) yang memakai email sari.lestari@student.test milik Sari Lestari. Error apa yang muncul dan pada kunci yang mana? Bedakan dengan error pada soal sebelumnya.	Perhatikan nama key pada pesan error: PRIMARY atau email?
8	Uji NOT NULL dengan dua cara: (a) isi kolom `nama` dengan NULL secara eksplisit; (b) tidak menyebut kolom `kode_prodi` sama sekali pada INSERT. Jalankan keduanya dan jelaskan mengapa pesan errornya berbeda.	Ada nilai NULL yang dikirim vs kolom wajib tanpa DEFAULT yang tidak disebut.
9	Uji NULL pada kolom UNIQUE: tambahkan dua mahasiswa baru yang keduanya tanpa email. Apakah keduanya diterima? Lalu hitung berapa mahasiswa yang emailnya NULL sekarang dan jelaskan mengapa NULL tidak dianggap "kembar".	Dua nilai NULL tidak dapat dikatakan sama; NULL berarti tidak diketahui.
10	Uji domain: masukkan mahasiswa dengan status_mhs "Drop-out" (NIM 25020099, nama "Uji Domain", 2025, SI1). Apa yang terjadi dan apa hubungannya dengan konsep domain pada relational model?	Domain = himpunan nilai yang sah. Apa domain kolom ENUM?
11	Bagian akademik meminta email mahasiswa menjadi wajib diisi. Ubah kolom `email` pada `mahasiswa` menjadi NOT NULL (tetap VARCHAR(80)). Apa yang terjadi pada rancangan dan data Anda saat ini? Tulis urutan langkah aman yang seharusnya ditempuh.	`ALTER TABLE ... MODIFY`. Apakah semua baris yang ada sudah punya email?

15. Challenge

Kerjakan setelah semua latihan selesai (nilai tambah, tidak ada kunci di modul):

Pasang foreign key lebih awal secara manual: hubungkan `dosen.kode_prodi` ke `program_studi.kode_prodi` dengan `ALTER TABLE ... ADD CONSTRAINT`. Beri nama constraint yang bermakna.

Lalu uji dua hal: (1) tambahkan dosen ber-kode prodi `ZZZ` yang tidak ada; (2) jelaskan mengapa foreign key baru bisa dipasang sekarang tetapi akan gagal bila ada dosen berkode prodi yang tidak terdaftar. Bandingkan dengan pengalaman Anda pada Latihan 11.

16. Expected Output

Praktikum dinyatakan berhasil bila:

- Tiga tabel `program_studi`, `dosen`, `mahasiswa` terbentuk pada `db_akademik` tanpa error yang tidak dijelaskan.

- `DESCRIBE` dan query `information_schema` menunjukkan PK pada kode_prodi/nidn/nim dan UNIQUE pada nama_prodi/email.
- Data valid (3 prodi, 2 dosen, 3 mahasiswa awal) berhasil masuk; status dan jabatan otomatis terisi DEFAULT.
- Tabel uji berisi pelanggaran PK (1062), UNIQUE (1062 pada key `email`), NOT NULL (1048, 1364), domain ENUM (1265), dan ALTER NOT NULL gagal (1265 di MariaDB / 1138 di MySQL) lengkap dengan interpretasinya.
- Berkas `praktikum_02.sql` berkomentar dan dapat dijalankan ulang dari database kosong.

17. Interpretation

Tulis jawaban singkat (2–4 kalimat, berdasarkan hasil **Anda sendiri**) pada laporan:

1. Apa perbedaan skema dan instance? Tunjukkan contoh keduanya dari pekerjaan Anda pada praktikum ini.
2. Mengapa `email` boleh NULL tetapi tetap ditandai UNIQUE? Apa dampaknya bagi dua mahasiswa yang sama-sama belum punya email?
3. Error 1062 muncul pada dua uji berbeda. Bagaimana Anda membedakan constraint mana yang dilanggar, dan mengapa membedakannya penting?
4. Mengapa NIM disimpan sebagai `CHAR`, bukan `INT`? Kolom mana pada rancangan Anda yang tipenya Anda ubah setelah melihat hasil uji, dan mengapa?

18. Troubleshooting

Pesan berikut dihasilkan dari eksekusi nyata pada MariaDB 10.11; teks pada MySQL 8.x bisa sedikit berbeda, tetapi nomor error umumnya sama.

Kode	Pemicu	Pesan	Penyebab & solusi
1064	`CREATE TABLE tabel_salah ( id INT NOT NULL, nama VARCHAR(20) NOT NULL, );`	1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near ')' at lin	Koma berlebih setelah definisi terakhir (atau kurung/kata kunci salah). Solusi: Hapus koma sebelum tanda kurung tutup; periksa juga jumlah kurung dan ejaan tipe data.
1050	`CREATE TABLE program_studi (kode CHAR(3));`	1050 (42S01): Table 'program_studi' already exists	Tabel bernama sama sudah ada di database aktif (mis. menjalankan berkas dua kali). Solusi: Pakai `DROP TABLE IF EXISTS` pada awal skrip pengembangan, atau `CREATE TABLE IF NOT EXISTS` bila tidak ingin mengubah struktur lama.
1068	`CREATE TABLE uji_pk (a INT PRIMARY KEY, b INT PRIMARY KEY);`	1068 (42000): Multiple primary key defined	Dua kolom masing-masing diberi PRIMARY KEY; tabel hanya boleh memiliki satu PK. Solusi: Untuk PK gabungan tulis satu klausa `PRIMARY KEY (a, b)` di akhir definisi.
1146	`SELECT * FROM prodi;`	1146 (42S02): Table 'db_akademik.prodi' doesn't exist	Nama tabel salah eja atau tabel belum dibuat di database aktif. Solusi: Periksa `SHOW TABLES;` dan `SELECT DATABASE();`; sesuaikan nama tabel.
1062	`INSERT INTO program_studi (kode_prodi, nama_prodi, fakultas) VALUES ('IF1', 'Nama Lain', 'Fakultas X');`	1062 (23000): Duplicate entry 'IF1' for key 'PRIMARY'	Nilai PK/UNIQUE yang dimasukkan sudah ada. Solusi: Gunakan nilai berbeda, atau ubah baris lama (bukan menambah baru). Baca nama key pada pesan.
1364	`INSERT INTO program_studi (kode_prodi, nama_prodi) VALUES ('XX1', 'Prodi Uji');`	1364 (HY000): Field 'fakultas' doesn't have a default value	Kolom NOT NULL tanpa DEFAULT (fakultas) tidak diisi. Solusi: Sebutkan nilai untuk semua kolom wajib, atau beri DEFAULT jika memang ada nilai bawaan yang masuk akal.
1406	`INSERT INTO program_studi (kode_prodi, nama_prodi, fakultas) VALUES ('INFO', 'Uji Panjang', 'Fakultas X');`	1406 (22001): Data too long for column 'kode_prodi' at row 1	Nilai lebih panjang daripada ukuran kolom (kode 4 karakter pada CHAR(3)). Solusi: Perbaiki datanya, atau ubah ukuran kolom bila kebutuhan memang berubah.

Kode	Pemicu	Pesan	Penyebab & solusi
1265	`INSERT INTO dosen (nidn, nama_dosen, jabatan, kode_prodi) VALUES ('0400000001', 'Uji Enum', 'Profesor', 'IF1');`	1265 (01000): Data truncated for column 'jabatan' at row 1	Nilai di luar daftar ENUM. Solusi: Gunakan nilai yang ada pada definisi ENUM; tambahkan nilai baru lewat ALTER TABLE bila memang sah.

19. Assessment

Bobot praktikum ini **1%** dari nilai akhir. Deliverable: `praktikum_02.sql` (CREATE TABLE + uji); tabel kolom rancangan beserta justifikasi tipe dan key; tangkapan layar DESCRIBE dan query information_schema; tabel uji constraint (skenario, error, interpretasi); jawaban interpretasi & refleksi.

Kriteria	Bobot	Indikator skor 4 (sangat baik)
Pemahaman konsep	20%	Membedakan skema/instance, candidate key/PK, dan domain dengan contoh dari pekerjaannya sendiri.
Database design	20%	Setiap kolom punya tipe, ukuran, NULL/NOT NULL, dan key yang beralasan sesuai kebutuhan data.
SQL correctness	30%	Ketiga CREATE TABLE benar dan dapat dijalankan ulang dari database kosong; sintaks rapi.
Testing	10%	Pengujian mencakup data valid dan seluruh pelanggaran (PK, UNIQUE, NOT NULL, domain) dengan bukti.
Interpretation	10%	Menjelaskan pesan error dan efeknya, termasuk perilaku NULL pada UNIQUE dan kegagalan ALTER.
Documentation	10%	Berkas .sql berkomentar, penamaan konsisten, laporan lengkap sesuai deliverable.

Skala 1–4 (1 belum tampak, 2 sebagian, 3 sesuai, 4 sangat baik dengan alasan). Skor praktikum = Σ(bobot × skor/4).

Checklist pengumpulan:

- Berkas `.sql` bernama `praktikum_02.sql` dan dapat dijalankan dari database kosong
- Ada tabel rancangan kolom sebelum kode
- Setiap pengujian error disertai kode dan interpretasi

20. Reflection

Jawab pada akhir laporan (3–5 kalimat):

1. Keputusan desain mana yang paling sulit (tipe, key, atau NULL) dan apa alasan akhirnya?
2. Constraint mana yang menurut Anda paling melindungi kualitas data? Berikan contoh kasus nyata.
3. Jika ada 100.000 mahasiswa, bagian rancangan mana yang akan Anda tinjau ulang?